

Triagent: Kompaktes technisches Design-Dokument

Zielbild, Programmablauf und aktueller Prototypstand für PIC GmbH

Ansprechpartner: Alexander Hülsmann und Tassilo Bechtolsheim

Adressat: Dennis Itzel, PIC GmbH **Stand:** April 2026

Einordnung

Dieses Dokument ist eine kompakte Arbeitsversion, keine vollständige Spezifikation. Es beschreibt das geplante Betriebsmodell von Triagent und reflektiert danach den heutigen Prototypstand aus technischer Sicht. Ziel ist, Entwicklungsaufwand, Integrationspunkte und offene Risiken besser einschätzen zu können.

1. Zielbild: Lebenszyklus von Triagent in der Praxis

Produktrolle im Zielbild

Triagent soll als Triage-Schicht zwischen gemeldeter verdächtiger E-Mail und finaler SOC- oder MSSP-Fallbearbeitung laufen. Das System soll kein Secure Email Gateway ersetzen, sondern nach der Meldung einer verdächtigen Mail strukturieren, vorsortieren, Evidenz sammeln und den Analysten beim Fallabschluss unterstützen.

Geplante Betriebsvarianten

- **On-prem / private deployment:** Betrieb in der Infrastruktur des Kunden oder in einer vom Kunden kontrollierten Umgebung. Wichtig für regulierte Umfelder und Teams mit Datenhoheitsanforderungen.
- **Cloud / managed deployment:** Betrieb als gehostete Variante für Teams, die weniger Betriebsaufwand möchten.
- **Hybrid-Option:** lokale Verarbeitung sensibler Artefakte mit optionalen externen Enrichment-Diensten, falls der Kunde diese freigibt.

Schnittstellen, die im Zielbild eingerichtet werden müssten

Mail-Intake	Outlook Add-in, manueller Upload von <code>.eml/.msg</code> , später optional Shared Mailbox, Microsoft Graph, IMAP oder API-basierte Weiterleitung.
Identität / Rollen	Lokale Benutzer und Rollen im Prototyp, später SSO / IdP, Gruppenmapping und Mandanten-/Kundenkontext für MSSPs.
Analyse-Engines	Aktivierbare Module für Header/Auth, URLs, Redirects, Anhänge, Lookalikes, Threat Intel, Sandbox-Anbindung und kundenspezifische Regeln.
Fallausgabe	PDF-/JSON-Fallbericht, IOC-CSV/JSON, später Ticketing- oder SIEM-Ausgabe, zum Beispiel Jira, ServiceNow, Splunk, Sentinel oder SOAR.
Audit / Compliance	Nachvollziehbarer Verlauf aus Intake, Analyse, Assist-Draft, Analystenentscheidung und Export.

Geplanter Programmablauf als Aktivitätsdiagramm in Textform

1. Mitarbeiter meldet verdächtige E-Mail über Outlook Add-in oder das SOC lädt eine `.eml/.msg`-Datei hoch.
2. Triagent speichert Originalnachricht und Anhänge unverändert als Artefakte.
3. Parser normalisieren Header, Body, URLs, Anhänge, Absenderdaten und Message-Metadaten.
4. Aktivierte Analyse-Engines erzeugen technische Signale, zum Beispiel SPF/DKIM/DMARC, URL-Redirects, Lookalikes, Attachment-Metadaten und Kontextindikatoren.
5. Triage-Policy ordnet den Fall einer Queue zu, zum Beispiel **Needs investigation**, **Likely benign** oder **Uncertain**.
6. Assist-Draft erzeugt einen Vorschlag für Disposition, Klassifikationscode, markierte Artefakte und Begründung.
7. Analyst prüft Evidenz, passt Vorschlag bei Bedarf an und bestätigt oder verwirft die Resolution.
8. Triagent schreibt Audit-Event, aktualisiert Fallstatus und erzeugt auf Wunsch PDF-/JSON-/IOC-Exporte.
9. Später optional: Übergabe an Ticketing, SIEM, SOAR oder MSSP-Kundensystem.

Erweiterbarkeit der Erkennungs- und Triage-Engines

Das Zielbild ist ein modulares Engine-Modell. Einzelne Engines sollen pro Deployment aktivierbar, deaktivierbar und konfigurierbar sein. Für Kunden wäre damit zum Beispiel steuerbar, ob externe URL-Auflösung, Sandbox-Abfragen, Threat-Intel-Lookups oder nur lokale Heuristiken erlaubt sind. Perspektivisch sollte jede Engine ein einheitliches Ergebnisformat liefern:

- `engine_id`, Version, Konfiguration
- Eingabeartefakte, zum Beispiel URL, Attachment, Header
- strukturierte Signale, Score-Beiträge und Begründungen
- Confidence, Fehlerstatus und Audit-Metadaten

2. Technische Reflexion: aktueller Prototypstand

Architektur des aktuellen Prototyps

Frontend	Next.js, React, TypeScript. Enthält Landing Page, Demo, Uploads, In-tray, Dashboard, Report Detail, Resolve Drawer, Settings und Admin-Sichten.
Backend	FastAPI in Python mit serviceorientierter Struktur für Parsing, Triage, Assist Drafts, Evidence Export, Audit, RBAC und Demo-Seeding.
Datenhaltung	PostgreSQL über SQLAlchemy und Alembic-Migrationen.
Objektspeicher	MinIO als S3-kompatibler Storage für Originalmails, Anhänge und Exporte.
Deployment	Docker Compose für lokalen und self-hosted Demo-Betrieb.
Client-Integration	Office.js Outlook Add-in mit API-Key-basierter Meldung an das Backend.

Komponentendiagramm in Textform

Outlook Add-in / Web UI / API → FastAPI Backend → Parser + Analyse-Services → Triage Policy + Assist Draft → PostgreSQL + MinIO → Report UI + Export + Audit

Was heute bereits implementiert ist

E-Mail-Intake	Upload und Parsing von .eml und .msg, Batch-Import synthetischer Demos, Outlook Add-in für gemeldete Mails.
Report-Workspace	Uploads, In-tray, Dashboard, Report Detail mit Details, Lookalikes, Authentication, URLs, Attachments, Transmission, X-Headers und Source.
Technische Analyse	SPF, DKIM, DMARC, ARC, URL-Extraktion, optionale Redirect-Auflösung, Same-org-Lookalike-Erkennung, Attachment-Metadaten, Raw Source Preservation.
Triage	Persistierte Triage-Buckets und Queue-Filter. Diese Logik ist bewusst als frühe Heuristik zu verstehen und noch nicht produktionsreif.
Assist Draft	Lokaler heuristischer Assist-Draft für Disposition, Klassifikation, Begründung und vorge-schlagene Artefakte. Aktuell Demo- und Prototypstand.
Resolution	Analyst-in-the-loop-Resolution mit Disposition, Klassifikationscode, Notiz und markierten Artefakten.
Exporte	Evidence Export als PDF, Markdown und JSON sowie IOC Export als CSV und JSON.
Governance	Session-RBAC, Rollen/Rechte, API Keys, Audit Events, Audit Export und Demo-User-Konzept.
Kampagnenbasis	Datenmodell und Backend-Mechaniken für Campaign Assignment, Recluster, Merge, Split und Campaign Evidence sind angelegt, aber noch nicht Produktfokus.

Pseudo-Algorithmus der aktuellen Triage

```
report = ingest(message)
preserve_original(report)
signals = parse_headers_urls_attachments(report)
signals += auth_summary(report)
signals += lookalike_detection(report)
signals += url_resolution(report)
bucket = triage_policy(signals, risk_score)
draft = assist_draft(signals, bucket)
analyst_reviews(report, signals, draft)
if analyst_confirms:
    write_resolution()
    write_audit_event()
    export_evidence_if_requested()
```

Bekannte technische Grenzen und Code-Risiken

- **Triage und Assist Draft sind noch frühe Heuristiken.** Sie funktionieren für die Demo- und synthetischen Fälle, brauchen aber reale Daten, Tuning und saubere Bewertungsmetriken.
- **Engine-Erweiterbarkeit ist noch nicht sauber abstrahiert.** Analysefunktionen existieren als Python-Services, aber noch nicht als stabiles Plugin- oder Engine-SDK.
- **Deployment ist Compose-first.** Für echten Betrieb fehlen noch Packaging, Secrets Management, Backup-/Restore-Automation, Observability und Update-Prozesse.
- **Integrationen sind noch begrenzt.** Outlook Add-in und Upload/API sind da, tiefe Ticketing-, SIEM-, SOAR- oder Sandbox-Integrationen sind noch offen.
- **SSO / Enterprise Auth ist nicht fertig.** Lokales RBAC existiert, LDAP-nahe Tests existieren, aber die vollständige Enterprise-IdP-Story ist offen.
- **Robustheit unter Last ist nicht validiert.** Es gibt lokale Builds und gezielte Backend-Tests, aber noch keine Lasttests, Penetrationstests oder formale Security Reviews.

3. Meilensteine, Teststand und nächste Schritte

Fertig oder weitgehend vorhanden

- lauffähiger Demo-Stack mit Frontend, Backend, Datenbank und Objektspeicher
- E-Mail-Parsing, Originalartefakt-Speicherung und technische Report-Sichten
- einfache Queue- und Triage-Logik
- Assist-Draft und analyst-in-the-loop-Resolution
- PDF-/JSON-/IOC-Exporte
- Audit- und RBAC-Grundlagen
- public read-only Demo und reproduzierbares Demo-Reset

Ausstehende Kernmeilensteine

- **M1: Engine-Konzept stabilisieren.** Einheitliches Interface, Aktivierung pro Deployment, Fehlerbehandlung, Versionierung und Audit-Kontext.
- **M2: Triage-Qualität validieren.** Reale oder anonymisierte Testdaten, Metriken für False Positives, False Negatives, Analysenzeit und Auto-Categorization-Rate.
- **M3: Integrationspfade priorisieren.** Outlook Add-in härten, danach Shared Mailbox / Graph oder Ticketing-Ausgabe entscheiden.
- **M4: Betriebshärtung.** Secrets, Backup, Monitoring, Logging, Update-Prozess, Mandantenfähigkeit und Deployment-Profil.
- **M5: Enterprise Auth.** SSO / IdP, Gruppenmapping, Rollenmodell und Audit-Anforderungen für Kundenumgebungen.
- **M6: Pilotfähigkeit.** Pilot-Runbook, Datenvereinbarung, Success Metrics, Support-Prozess und klare Grenzen der Automatisierung.

Optionale spätere Meilensteine

- Kampagnen-Workflow als Hauptoberfläche für wiederkehrende Wellen
- Mandantenfähige MSSP-Sichten mit Kundenkontext
- Sandbox- und Threat-Intel-Adapter
- Case-Management- und SIEM-Integrationen
- Cloud-Angebot neben on-prem / private deployment
- Lern- und Feedbackschleife aus bestätigten Analystenentscheidungen

Teststand

Es existieren gezielte Backend-Tests für Parsing, Auth-Zusammenfassung, URL-Resolution, Lookalike-Erkennung, Triage, Assist-Drafts, Evidence Exports, synthetischen Korpus und LDAP-nahe Rollenzuordnung. Zusätzlich wird das Frontend regelmäßig über den Next.js-Build geprüft. Es gibt aber aktuell **keine belastbare Coverage-Zahl**, keine vollständigen UI-Regressionstests für alle Workflows und keine formalen Last- oder Security-Tests. Der Code ist damit **demo- und pilotfähig**, aber noch nicht als robustes Enterprise-Produkt einzustufen.

Einschätzung für PIC GmbH

Der sinnvollste Beitrag eines technischen Partners läge aus unserer Sicht in drei Bereichen:

- **Engineering:** Engine-Abstraktion, Deployment-Härtung, Integrationen und Qualitätssicherung.
- **Evaluierung:** Definition realistischer Testfälle, Messung der Triage-Qualität und Bewertung der Analysten-Workflows.
- **Vertrieb / Partnering:** Einschätzung, welche Betriebsvariante und welcher erste Integrationspfad für SOC's oder MSSP's am besten verkaufbar ist.

Kurzfazit

Triagent hat bereits einen funktionsfähigen Kern für reported-email triage, Evidence Review, Assist Drafts, Resolution und Exporte. Die größten offenen Arbeiten liegen nicht in einer weiteren Feature-Demo, sondern in Produktisierung, Engine-Erweiterbarkeit, Integrationen, Betriebshärtung und echter Validierung auf realistischen Workflows.